

BELAJAR PYTHON UNTUK PEMULA

Dari dasar-dasar pemrograman hingga membuat program Python sederhana



Python menerjemahkan dan menjalankan instruksi program.

Penulis: HANZZY DEV
Website: hanie-alfarisy-dev.vercel.app

Edisi Pemula • 2026

Tentang Ebook Ini

Ebook ini dibuat untuk pembaca yang benar-benar baru mengenal Python. Urutannya sengaja dibuat bertahap: kenalan dengan pemrograman, mengenal Python, memahami sintaks dasar, mengolah data, membuat logika, mengulang proses, memakai fungsi, membaca/menulis file, menangani error, sampai mengenal konsep lanjutan.

Setiap bab berisi materi terlebih dahulu. Setelah semua materi bab selesai, barulah diberikan 5 latihan. Jadi kamu tidak perlu berhenti untuk mengerjakan soal setiap kali satu konsep baru muncul.

Sumber rujukan

Penjelasan teknis dalam ebook ini diselaraskan dengan dokumentasi resmi Python 3.14, yang mencakup interpreter, sintaks dasar, control flow, struktur data, modul, input/output, error, class, standard library, serta virtual environment dan package. Dokumentasi resmi Python juga menjelaskan bahwa Python merupakan bahasa pemrograman yang mudah dipelajari dengan struktur data tingkat tinggi dan dukungan pengembangan aplikasi yang luas. [\[cite turn0view0\]](#)

Website penulis

HANZZY DEV — hanie-alfarisy-dev.vercel.app

Daftar Isi

- Bab 1 — Kenalan dengan Pemrograman dan Python
- Bab 2 — Menyiapkan Python dan Menjalankan Program
- Bab 3 — Sintaks Dasar, Komentar, dan Output
- Bab 4 — Variabel dan Tipe Data
- Bab 5 — Operator dan Ekspresi
- Bab 6 — Input dan Konversi Data
- Bab 7 — Percabangan if, elif, else
- Bab 8 — Perulangan for dan while
- Bab 9 — String dan Pengolahan Teks
- Bab 10 — List, Tuple, Set, dan Dictionary
- Bab 11 — Fungsi dan Modularisasi
- Bab 12 — File, JSON, dan Data Sederhana
- Bab 13 — Error dan Exception Handling
- Bab 14 — Modul, Package, pip, dan Virtual Environment
- Bab 15 — Object-Oriented Programming (OOP)
- Bab 16 — Membuat Program Python Sederhana
- Bab 17 — Debugging, Style, dan Kebiasaan Coding
- Bab 18 — Peta Jalan Belajar Python Selanjutnya

Kenalan dengan Pemrograman dan Python

Sebelum menulis kode, kita perlu memahami cara berpikir yang digunakan programmer. Program pada dasarnya adalah sekumpulan instruksi yang disusun agar komputer melakukan pekerjaan tertentu. Python adalah salah satu bahasa yang membuat penulisan instruksi tersebut relatif mudah dibaca.



Python menerjemahkan dan menjalankan instruksi program.

Apa itu program?

Program adalah instruksi terurut yang memberi tahu komputer apa yang harus dilakukan. Contoh paling sederhana adalah instruksi untuk menampilkan teks. Dari konsep ini, kita kemudian berkembang ke variabel, kondisi, perulangan, fungsi, dan struktur data.

Apa itu Python?

Python adalah bahasa pemrograman tingkat tinggi dengan sintaks yang dirancang agar relatif mudah dibaca. Dokumentasi resmi Python menyebut Python sebagai bahasa yang mudah dipelajari dan memiliki struktur data tingkat tinggi yang kuat. [\[cite\]turn0view0](#)

Mengapa Python populer?

Python digunakan pada banyak bidang, termasuk scripting, otomasi, web, analisis data, dan pembelajaran mesin. Ekosistem library dan dokumentasinya juga membantu pengembang menyelesaikan pekerjaan tanpa selalu membuat semuanya dari nol.

Cara berpikir algoritmik

Algoritma adalah urutan langkah untuk menyelesaikan masalah. Sebelum coding, biasanya memecah masalah menjadi input, proses, dan output.

Interpreter dan source code

Source code adalah kode yang ditulis programmer. Interpreter Python menjalankan kode tersebut sehingga instruksi dapat dieksekusi oleh komputer.

Latihan Bab 1

Kerjakan setelah memahami seluruh materi bab ini. Cobalah menulis kode sendiri sebelum melihat referensi lain.

1. Jelaskan dengan kata-katamu sendiri apa yang dimaksud program.
2. Sebutkan tiga alasan Python cocok untuk pemula.
3. Apa perbedaan algoritma dan program?

4. 4. Buat algoritma sederhana untuk menghitung luas persegi panjang.
5. 5. Tulis tiga bidang yang menurutmu bisa menggunakan Python.

Menyiapkan Python dan Menjalankan Program

Ada banyak cara menjalankan Python. Yang penting bagi pemula adalah memahami hubungan antara editor, file program, dan Python interpreter. Setelah itu, proses mencoba kode menjadi jauh lebih mudah.

Memilih lingkungan kerja

Kamu bisa menggunakan editor kode di komputer atau lingkungan lain yang menyediakan Python. Untuk pemula, pilih lingkungan yang membuat kamu mudah menjalankan file dan melihat pesan error.

File .py

Kode Python biasanya disimpan dengan ekstensi .py. Misalnya, program pertama dapat disimpan sebagai halo.py.

Mode interaktif

Mode interaktif memungkinkan kamu mengetik satu perintah lalu langsung melihat hasilnya. Ini bagus untuk mencoba konsep kecil dengan cepat.

Menjalankan program

Saat file .py dijalankan, Python membaca instruksi di dalamnya sesuai aturan sintaks dan menjalankan program.

Kesalahan awal yang umum

Kesalahan yang sering terjadi adalah salah ketik nama variabel, tanda kurung tidak lengkap, indentasi salah, atau mencampur tipe data secara tidak tepat.

Latihan Bab 2

Kerjakan setelah memahami seluruh materi bab ini. Cobalah menulis kode sendiri sebelum melihat referensi lain.

1. 1. Buat file bernama halo.py dan jalankan.
2. 2. Tulis program yang menampilkan nama dan kelasmu.
3. 3. Jelaskan perbedaan mode interaktif dan file .py.
4. 4. Apa yang terjadi jika tanda kutip pada print() tidak ditutup?
5. 5. Buat program yang menampilkan tiga baris teks berbeda.

Sintaks Dasar, Komentar, dan Output

Sintaks adalah aturan penulisan kode. Python terkenal dengan sintaks yang relatif ringkas, tetapi tetap memiliki aturan seperti penggunaan titik dua, tanda kurung, dan indentasi.

print()

Fungsi print() digunakan untuk menampilkan informasi ke output. Contoh: print("Halo, Python!").

```
print("Halo, Python!")  
print("Saya sedang belajar coding.")
```

Komentar satu baris

Komentar dimulai dengan # dan tidak dijalankan sebagai instruksi program. Komentar sebaiknya membantu menjelaskan alasan atau bagian penting kode.

Indentasi

Indentasi adalah spasi di awal baris. Python menggunakan indentasi untuk menunjukkan blok kode pada kondisi, loop, fungsi, dan struktur lainnya.

String sederhana

String adalah teks yang ditulis di antara tanda kutip tunggal atau ganda, misalnya 'Python' atau "Python".

Escape character dan f-string pengantar

Escape sequence seperti \n dapat membuat baris baru. f-string memungkinkan kita memasukkan nilai variabel ke dalam teks dengan cara yang nyaman.

Latihan Bab 3

Kerjakan setelah memahami seluruh materi bab ini. Cobalah menulis kode sendiri sebelum melihat referensi lain.

1. Buat output biodata minimal lima baris.
2. Tambahkan komentar yang menjelaskan setiap bagian program.
3. Tampilkan kalimat yang mengandung tanda kutip.
4. Buat output dengan f-string yang memuat nama dan umur.
5. Jelaskan mengapa indentasi penting dalam Python.

Ringkasan Bab 3

Gunakan halaman ini sebagai checkpoint. Jangan hanya menghafal sintaks; pastikan kamu bisa menjelaskan kapan sebuah konsep digunakan dan apa masalah yang diselesaikannya.

- Pahami konsep sebelum menghafal kode.
- Coba ubah contoh dengan nilai dan nama yang berbeda.
- Baca pesan error sebelum mencari solusi.
- Biasakan menulis kode sedikit demi sedikit lalu menjalankannya.
- Simpan proyek kecil agar perkembanganmu terlihat.

Catatan pribadi

Tulis konsep yang paling kamu pahami, konsep yang masih membingungkan, dan satu ide program yang ingin kamu coba.

Variabel dan Tipe Data

Variabel dapat dibayangkan sebagai nama yang kita gunakan untuk merujuk pada suatu nilai. Python memiliki beberapa tipe data bawaan yang sangat sering digunakan.



Variabel adalah nama yang menunjuk ke sebuah nilai.

Konsep variabel

Variabel mengikat sebuah nama ke nilai. Misalnya `nama = "Alya"` membuat `nama` merujuk pada string tersebut.

int, float, str, bool

`int` menyimpan bilangan bulat, `float` bilangan desimal, `str` teks, dan `bool` nilai `True` atau `False`.

None

`None` mewakili ketiadaan nilai. Ini berbeda dari `0`, string kosong, atau `False`.

type()

Fungsi `type()` dapat membantu melihat tipe sebuah nilai saat belajar atau debugging.

Penamaan variabel

Gunakan nama yang jelas, misalnya `total_harga` daripada `x`. Hindari nama yang membingungkan dan kata kunci Python.

Latihan Bab 4

Kerjakan setelah memahami seluruh materi bab ini. Cobalah menulis kode sendiri sebelum melihat referensi lain.

1. Buat empat variabel untuk nama, umur, tinggi, dan status siswa.
2. Tampilkan tipe data masing-masing variabel.
3. Ubah nilai sebuah variabel lalu tampilkan hasilnya.
4. Tentukan tipe data yang tepat untuk harga barang dan jumlah barang.
5. Sebutkan tiga contoh nama variabel yang baik dan tiga yang sebaiknya dihindari.

Operator dan Ekspresi

Operator memungkinkan program melakukan perhitungan, membandingkan nilai, dan menggabungkan kondisi. Memahami operator adalah dasar untuk membuat program yang mengambil keputusan.

Aritmetika

Operator `+`, `-`, `*`, `/`, `//`, `%`, dan `**` digunakan untuk operasi matematika umum.

Perbandingan

Operator `==`, `!=`, `<`, `>`, `<=`, dan `>=` menghasilkan nilai boolean.

Logika

`and`, `or`, dan `not` digunakan untuk menggabungkan atau membalik kondisi.

Assignment

Operator assignment seperti `=` dan `+=` digunakan untuk memberikan atau memperbarui nilai variabel.

Prioritas operasi

Python mengikuti aturan prioritas operasi. Gunakan tanda kurung jika ingin membuat urutan perhitungan lebih jelas.

Latihan Bab 5

Kerjakan setelah memahami seluruh materi bab ini. Cobalah menulis kode sendiri sebelum melihat referensi lain.

1. Hitung luas dan keliling persegi panjang menggunakan operator aritmetika.
2. Buat ekspresi yang mengecek apakah umur lebih besar atau sama dengan 17.
3. Gunakan `and`/`or`/`not` dalam tiga contoh kondisi.
4. Jelaskan perbedaan `/`, `//`, dan `%`.
5. Buat program penghitung diskon sederhana.

Input dan Konversi Data

Program yang menerima input bisa berinteraksi dengan pengguna. Karena `input()` menghasilkan teks, kita juga perlu memahami konversi tipe data.

`input()`

`input()` membaca teks yang dimasukkan pengguna dan mengembalikannya sebagai string.

String dari input

Walaupun pengguna mengetik angka, hasil `input()` tetap berupa string sampai kita mengonversinya.

`int()` dan `float()`

`int()` dan `float()` digunakan untuk mengubah string numerik menjadi angka sesuai kebutuhan.

```
umur = int(input("Umur: "))
tinggi = float(input("Tinggi: "))
print(umur, tinggi)
```

Validasi sederhana

Validasi memastikan input sesuai yang diharapkan, misalnya memastikan angka tidak negatif sebelum dipakai.

Input sebagai bagian dari program interaktif

Dengan input, program dapat meminta informasi pengguna lalu menggunakan informasi tersebut dalam proses.

Latihan Bab 6

Kerjakan setelah memahami seluruh materi bab ini. Cobalah menulis kode sendiri sebelum melihat referensi lain.

1. Minta pengguna memasukkan nama lalu sapa.
2. Minta panjang dan lebar lalu hitung luas.
3. Minta dua angka lalu hitung jumlahnya.
4. Jelaskan mengapa input angka biasanya perlu dikonversi.
5. Buat program menghitung total harga dari harga satuan dan jumlah barang.

Ringkasan Bab 6

Gunakan halaman ini sebagai checkpoint. Jangan hanya menghafal sintaks; pastikan kamu bisa menjelaskan kapan sebuah konsep digunakan dan apa masalah yang diselesaikannya.

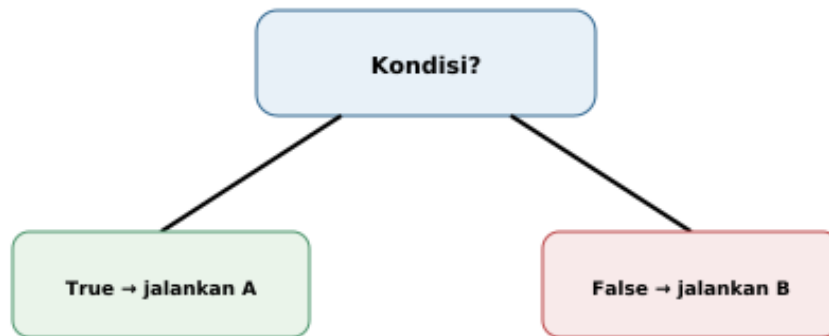
- Pahami konsep sebelum menghafal kode.
- Coba ubah contoh dengan nilai dan nama yang berbeda.
- Baca pesan error sebelum mencari solusi.
- Biasakan menulis kode sedikit demi sedikit lalu menjalankannya.
- Simpan proyek kecil agar perkembanganmu terlihat.

Catatan pribadi

Tulis konsep yang paling kamu pahami, konsep yang masih membingungkan, dan satu ide program yang ingin kamu coba.

Percabangan if, elif, else

Percabangan membuat program dapat memilih tindakan berdasarkan kondisi. Inilah salah satu langkah penting dari program sederhana menuju program yang lebih 'cerdas'.



Konsep kondisi

Kondisi adalah ekspresi yang menghasilkan True atau False dan menentukan jalur program.

if

Blok if dijalankan ketika kondisi bernilai True.

```
nilai = 80
if nilai >= 75:
    print("Lulus")
```

else

else menjadi pilihan ketika kondisi if tidak terpenuhi.

elif

elif memungkinkan kita memeriksa beberapa kondisi secara berurutan.

Kondisi bertingkat

if dapat berada di dalam if lain, tetapi jangan membuat struktur terlalu dalam jika bisa dibuat lebih sederhana.

Truthiness pengantar

Beberapa nilai dianggap False dalam konteks kondisi, seperti False, None, 0, dan string kosong.

Latihan Bab 7

Kerjakan setelah memahami seluruh materi bab ini. Cobalah menulis kode sendiri sebelum melihat referensi lain.

1. Buat program yang menentukan bilangan positif atau negatif.
2. Buat program kategori nilai: A, B, C, D berdasarkan batas yang kamu tentukan.
3. Buat pengecekan apakah sebuah angka genap atau ganjil.
4. Gunakan elif untuk memilih menu.

5. 5. Buat program sederhana yang menentukan boleh/tidak berdasarkan dua kondisi.

Perulangan for dan while

Perulangan digunakan ketika pekerjaan yang sama perlu dilakukan berkali-kali. Dengan loop, kita tidak perlu menulis instruksi yang sama berulang-ulang.



for/while membuat program mengulang pekerjaan secara terkontrol.

Mengapa perlu loop?

Tanpa loop, pekerjaan berulang membuat kode panjang. Loop membuat pola pengulangan menjadi ringkas.

for

for cocok untuk mengulangi item dalam iterable seperti list atau rentang angka.

```
for i in range(1, 6):  
    print(i)
```

range()

range() menghasilkan urutan angka yang sering digunakan bersama for.

while

while mengulang selama kondisi tetap True. Pastikan kondisi dapat berubah agar loop berhenti.

```
angka = 1  
while angka <= 5:  
    print(angka)  
    angka += 1
```

break dan continue

break menghentikan loop, sedangkan continue melewati sisa iterasi saat ini.

Menghindari infinite loop

Infinite loop terjadi ketika kondisi while tidak pernah menjadi False. Selalu periksa perubahan variabel pengendali.

Latihan Bab 8

Kerjakan setelah memahami seluruh materi bab ini. Cobalah menulis kode sendiri sebelum melihat referensi lain.

1. Tampilkan angka 1 sampai 10 menggunakan for.

2. 2. Hitung jumlah angka 1 sampai 100.
3. 3. Buat tabel perkalian sebuah angka.
4. 4. Buat while yang meminta input sampai pengguna mengetik selesai.
5. 5. Jelaskan kapan for lebih cocok daripada while.

String dan Pengolahan Teks

Teks adalah data yang sangat sering dipakai: nama, pesan, judul, alamat, dan sebagainya. Python menyediakan banyak operasi string untuk memprosesnya.

Index dan slicing

String memiliki indeks yang dimulai dari 0. Slicing mengambil sebagian karakter, misalnya `teks[0:3]`.

`len()`

`len()` mengembalikan jumlah karakter dalam string atau jumlah item pada beberapa struktur data.

`lower/upper`

`lower()` dan `upper()` mengubah kapitalisasi teks.

`strip`

`strip()` membantu menghapus spasi kosong di bagian awal dan akhir string.

`replace`

`replace()` mengganti bagian teks dengan teks lain.

`split` dan `join`

`split()` memecah string menjadi bagian-bagian, sedangkan `join()` menggabungkan iterable string dengan pemisah.

`f-string`

Konsep ini perlu dipahami melalui contoh kode dan latihan bertahap. Fokus pada satu ide utama, lalu coba ubah contoh agar sesuai dengan kebutuhanmu sendiri.

Latihan Bab 9

Kerjakan setelah memahami seluruh materi bab ini. Cobalah menulis kode sendiri sebelum melihat referensi lain.

1. Hitung panjang sebuah nama.
2. Ubah kalimat menjadi huruf kecil dan huruf besar.
3. Ambil tiga karakter pertama dari sebuah string.
4. Ganti kata tertentu dengan kata lain menggunakan `replace()`.
5. Buat program yang memecah kalimat menjadi daftar kata.

Ringkasan Bab 9

Gunakan halaman ini sebagai checkpoint. Jangan hanya menghafal sintaks; pastikan kamu bisa menjelaskan kapan sebuah konsep digunakan dan apa masalah yang diselesaikannya.

- Pahami konsep sebelum menghafal kode.
- Coba ubah contoh dengan nilai dan nama yang berbeda.
- Baca pesan error sebelum mencari solusi.
- Biasakan menulis kode sedikit demi sedikit lalu menjalankannya.
- Simpan proyek kecil agar perkembanganmu terlihat.

Catatan pribadi

Tulis konsep yang paling kamu pahami, konsep yang masih membingungkan, dan satu ide program yang ingin kamu coba.

List, Tuple, Set, dan Dictionary

Struktur data membantu kita menyimpan kumpulan informasi. List, tuple, set, dan dictionary memiliki karakteristik berbeda sehingga pilihan struktur harus disesuaikan dengan kebutuhan.

list	[10, 20, 30]
tuple	(10, 20, 30)
set	{10, 20, 30}
dict	{"nama": "Ana"}

List

List adalah koleksi berurutan yang dapat diubah. Contoh: angka = [10, 20, 30].

```
buah = ["apel", "mangga", "jeruk"]
buah.append("pisang")
print(buah)
```

Tuple

Tuple mirip list tetapi biasanya digunakan untuk kumpulan nilai yang tidak dimaksudkan untuk diubah.

Set

Set menyimpan nilai unik dan tidak menggunakan indeks seperti list.

Dictionary

Dictionary menyimpan pasangan key-value, misalnya {"nama": "Raka"}.

```
siswa = {"nama": "Budi", "kelas": "X"}
print(siswa["nama"])
```

Indexing

List dan tuple dapat diakses berdasarkan indeks. Dictionary diakses menggunakan key.

Method dasar

Struktur data memiliki method yang membantu menambah, menghapus, atau memproses data.

Memilih struktur data

Pilih struktur data berdasarkan kebutuhan: urutan, keunikan, akses berdasarkan key, atau data yang tidak perlu diubah.

Latihan Bab 10

Kerjakan setelah memahami seluruh materi bab ini. Cobalah menulis kode sendiri sebelum melihat referensi lain.

1. 1. Buat list berisi lima nama teman.
2. 2. Tambahkan dan hapus satu item dari list.
3. 3. Buat tuple berisi koordinat x dan y.
4. 4. Buat set dari daftar angka yang memiliki duplikat.
5. 5. Buat dictionary biodata dan tampilkan salah satu nilainya.

Fungsi dan Modularisasi

Fungsi membantu kita mengelompokkan kode yang melakukan satu pekerjaan tertentu. Kode menjadi lebih mudah digunakan ulang, diuji, dan dirawat.



Mengapa fungsi?

Fungsi mengurangi pengulangan dan membuat program lebih terorganisir.

def

Kata kunci def digunakan untuk mendefinisikan fungsi.

```
def sapa(nama):  
    return f"Halo, {nama}!"  
  
print(sapa("Budi"))
```

Parameter

Parameter adalah nama yang menerima nilai saat fungsi dipanggil.

Return

return mengirimkan hasil dari fungsi kepada kode yang memanggilnya.

Default parameter

Parameter dapat memiliki nilai default sehingga pemanggil tidak selalu harus memberikan argumen.

Scope pengantar

Scope menentukan bagian program yang dapat mengakses sebuah nama.

Membuat kode reusable

Fungsi yang baik fokus pada satu pekerjaan dan dapat dipakai kembali.

Latihan Bab 11

Kerjakan setelah memahami seluruh materi bab ini. Cobalah menulis kode sendiri sebelum melihat referensi lain.

1. Buat fungsi menyapa nama.
2. Buat fungsi menghitung luas persegi.
3. Buat fungsi yang menerima dua angka dan mengembalikan nilai terbesar.

4. 4. Buat fungsi dengan default parameter.
5. 5. Jelaskan manfaat memecah program besar menjadi beberapa fungsi.

File, JSON, dan Data Sederhana

Program nyata sering perlu menyimpan informasi di luar memori sementara. Python dapat membaca dan menulis file teks serta format terstruktur seperti JSON.

Membuka file

Python dapat membuka file menggunakan `open()`. Gunakan `with` agar file ditutup dengan aman setelah selesai.

Mode r/w/a

r untuk membaca, w untuk menulis ulang, dan a untuk menambahkan isi.

with open

Pola `with open(...)` as file membantu mengelola file secara aman dan otomatis menutupnya setelah blok selesai.

```
with open("catatan.txt", "w", encoding="utf-8") as file:  
    file.write("Belajar Python")
```

Menulis file teks

Method `write()` dapat menulis teks ke file.

Membaca file

`read()` membaca seluruh isi, sedangkan `readline()` atau iterasi dapat digunakan untuk membaca bertahap.

JSON pengantar

JSON adalah format teks terstruktur yang umum dipakai untuk pertukaran data. Python menyediakan modul `json` untuk bekerja dengan JSON.

Latihan Bab 12

Kerjakan setelah memahami seluruh materi bab ini. Cobalah menulis kode sendiri sebelum melihat referensi lain.

1. Buat file `catatan.txt` lalu tulis satu kalimat.
2. Baca isi file dan tampilkan ke layar.
3. Tambahkan teks tanpa menghapus isi lama.
4. Buat data dictionary lalu ubah menjadi JSON.
5. Jelaskan perbedaan file teks biasa dan JSON secara sederhana.

Ringkasan Bab 12

Gunakan halaman ini sebagai checkpoint. Jangan hanya menghafal sintaks; pastikan kamu bisa menjelaskan kapan sebuah konsep digunakan dan apa masalah yang diselesaikannya.

- Pahami konsep sebelum menghafal kode.
- Coba ubah contoh dengan nilai dan nama yang berbeda.
- Baca pesan error sebelum mencari solusi.
- Biasakan menulis kode sedikit demi sedikit lalu menjalankannya.
- Simpan proyek kecil agar perkembanganmu terlihat.

Catatan pribadi

Tulis konsep yang paling kamu pahami, konsep yang masih membingungkan, dan satu ide program yang ingin kamu coba.

Error dan Exception Handling

Error adalah bagian normal dari proses pemrograman. Programmer yang baik bukan programmer yang tidak pernah mendapat error, tetapi programmer yang bisa membaca dan menanganinya.

SyntaxError

SyntaxError muncul ketika kode tidak mengikuti aturan sintaks Python.

NameError

NameError sering terjadi saat nama yang digunakan belum didefinisikan atau salah ketik.

TypeError

TypeError muncul ketika operasi tidak sesuai dengan tipe data yang digunakan.

ValueError

ValueError terjadi ketika tipe yang benar diberikan tetapi nilainya tidak sesuai untuk operasi tersebut.

try dan except

try menjalankan kode yang mungkin gagal, sedangkan except menangani exception tertentu.

```
try:
    angka = int(input("Angka: "))
    print(10 / angka)
except ValueError:
    print("Input harus berupa angka.")
except ZeroDivisionError:
    print("Tidak boleh membagi dengan nol.")
```

else dan finally

else dapat dijalankan jika tidak ada exception; finally dijalankan setelah proses try/except selesai.

Pesan error sebagai petunjuk

Traceback biasanya memberi lokasi dan jenis error. Membacanya pelan-pelan adalah bagian dari debugging.

Latihan Bab 13

Kerjakan setelah memahami seluruh materi bab ini. Cobalah menulis kode sendiri sebelum melihat referensi lain.

1. Buat contoh program yang menghasilkan ValueError lalu tangani.
2. Tangani pembagian dengan input yang tidak valid.
3. Jelaskan perbedaan error sintaks dan exception saat runtime.
4. Gunakan try-except-else pada program sederhana.
5. Mengapa pesan error sebaiknya dibaca sebelum memperbaiki kode?

Modul, Package, pip, dan Virtual Environment

Saat program membesar, kode perlu dibagi menjadi modul dan package. Python juga memiliki pip untuk mengelola package dan virtual environment untuk memisahkan lingkungan proyek.

Apa itu module?

Module adalah file Python yang berisi kode yang dapat digunakan dari file lain.

import

import memasukkan module ke namespace program sehingga fungsi atau objeknya dapat digunakan.

```
import math
print(math.sqrt(25))
```

from ... import ...

Sintaks ini mengambil nama tertentu dari sebuah module.

Standard library

Python menyediakan banyak module bawaan seperti math, random, pathlib, datetime, json, dan os.

Package

Package mengelompokkan module sehingga proyek atau library dapat disusun lebih rapi.

pip

pip adalah alat untuk memasang dan mengelola package Python dari ekosistem package yang tersedia.

Virtual environment

Virtual environment membuat lingkungan package yang terpisah untuk sebuah proyek, sehingga dependensi tidak mudah saling mengganggu.

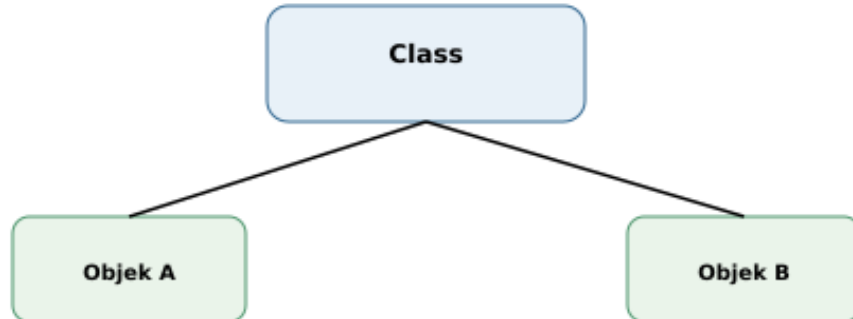
Latihan Bab 14

Kerjakan setelah memahami seluruh materi bab ini. Cobalah menulis kode sendiri sebelum melihat referensi lain.

1. 1. Gunakan modul math untuk menghitung akar atau pembulatan.
2. 2. Buat file module sendiri berisi satu fungsi lalu import dari file lain.
3. 3. Jelaskan fungsi pip.
4. 4. Jelaskan mengapa virtual environment berguna.
5. 5. Sebutkan tiga modul standard library yang kamu ketahui dan kegunaannya.

Object-Oriented Programming (OOP)

OOP adalah cara menyusun program menggunakan class dan object. Konsep ini banyak ditemui pada proyek yang lebih besar dan library Python.



Class adalah cetak biru; objek adalah hasil pembuatannya.

Class dan object

Class adalah rancangan, sedangkan object adalah instance yang dibuat dari class.

```
class Siswa:
    def __init__(self, nama):
        self.nama = nama

s1 = Siswa("Budi")
print(s1.nama)
```

Attribute

Attribute adalah data yang dimiliki object, misalnya nama dan umur.

Method

Method adalah fungsi yang didefinisikan di dalam class dan biasanya bekerja dengan data object.

__init__

__init__ biasanya digunakan untuk menginisialisasi attribute saat object dibuat.

self

self merujuk pada instance yang sedang digunakan dalam method.

Encapsulation pengantar

Encapsulation adalah pengorganisasian data dan perilaku dalam object serta pengendalian bagaimana bagian tersebut digunakan.

Inheritance pengantar

Inheritance memungkinkan class turunan mewarisi perilaku dari class induk.

Latihan Bab 15

Kerjakan setelah memahami seluruh materi bab ini. Cobalah menulis kode sendiri sebelum melihat referensi lain.

1. 1. Buat class Siswa dengan nama dan kelas.
2. 2. Tambahkan method untuk menampilkan biodata.
3. 3. Buat class Persegi dengan method luas.
4. 4. Jelaskan perbedaan class dan object.
5. 5. Buat contoh sederhana inheritance dengan class induk dan turunan.

Ringkasan Bab 15

Gunakan halaman ini sebagai checkpoint. Jangan hanya menghafal sintaks; pastikan kamu bisa menjelaskan kapan sebuah konsep digunakan dan apa masalah yang diselesaikannya.

- Pahami konsep sebelum menghafal kode.
- Coba ubah contoh dengan nilai dan nama yang berbeda.
- Baca pesan error sebelum mencari solusi.
- Biasakan menulis kode sedikit demi sedikit lalu menjalankannya.
- Simpan proyek kecil agar perkembanganmu terlihat.

Catatan pribadi

Tulis konsep yang paling kamu pahami, konsep yang masih membingungkan, dan satu ide program yang ingin kamu coba.

Membuat Program Python Sederhana

Sekarang kita gabungkan beberapa konsep dasar menjadi program yang benar-benar bisa digunakan. Proyek kecil mengajarkan cara menyusun kode sebagai satu kesatuan.

Merancang kebutuhan

Sebelum coding, tulis tujuan program, input, proses, output, dan aturan yang dibutuhkan.

Membuat menu

Menu teks dapat dibuat menggunakan print dan input, lalu diproses dengan percabangan.

Memisahkan fungsi

Operasi yang berbeda sebaiknya dipisahkan ke fungsi agar program mudah dibaca.

Validasi input

Program perlu menghadapi input yang tidak sesuai agar tidak langsung berhenti.

Menyimpan data

Data sederhana dapat disimpan di file teks atau JSON.

Contoh proyek: kalkulator mini

Kalkulator mini cocok untuk menggabungkan input, operator, fungsi, percabangan, loop, dan exception handling.

Latihan Bab 16

Kerjakan setelah memahami seluruh materi bab ini. Cobalah menulis kode sendiri sebelum melihat referensi lain.

1. 1. Buat kalkulator dua angka dengan menu +, -, *, /.
2. 2. Tambahkan validasi pembagian dengan nol.
3. 3. Pisahkan setiap operasi menjadi fungsi.
4. 4. Tambahkan perulangan agar program bisa digunakan berkali-kali.
5. 5. Kembangkan program menjadi kalkulator mini yang memiliki menu keluar.

Debugging, Style, dan Kebiasaan Coding

Kemampuan memperbaiki program sama pentingnya dengan kemampuan menulisnya. Debugging, penamaan yang jelas, testing, dan style akan membuat kode lebih mudah dipahami.

Debugging

Debugging adalah proses mencari penyebab kesalahan dan memperbaikinya.

Membaca traceback

Traceback menunjukkan jejak pemanggilan dan biasanya menunjukkan baris yang memicu exception.

Nama variabel jelas

Nama yang jelas mengurangi kebutuhan komentar dan memudahkan orang lain memahami kode.

PEP 8 pengantar

PEP 8 adalah panduan style Python yang membantu membuat kode lebih konsisten dan mudah dibaca.

Komentar yang berguna

Komentar sebaiknya menjelaskan hal yang tidak langsung terlihat dari kode, bukan sekadar mengulang kode.

Testing sederhana

Test case adalah contoh input dan hasil yang digunakan untuk memastikan program bekerja sesuai harapan.

Refactoring

Refactoring memperbaiki struktur kode tanpa mengubah tujuan perilaku program.

Latihan Bab 17

Kerjakan setelah memahami seluruh materi bab ini. Cobalah menulis kode sendiri sebelum melihat referensi lain.

1. Temukan dan perbaiki tiga kesalahan pada program buatanmu sendiri.
2. Ubah nama variabel yang terlalu singkat menjadi lebih jelas.
3. Tambahkan satu fungsi kecil untuk mengurangi kode berulang.
4. Buat tiga test case untuk kalkulator mini.
5. Jelaskan langkah debugging yang akan kamu lakukan saat program tidak berjalan.

Peta Jalan Belajar Python Selanjutnya

Belajar Python tidak berhenti di sintaks dasar. Setelah fondasi kuat, kamu bisa memilih jalur sesuai minat dan membuat proyek yang semakin nyata.

Level setelah dasar

Setelah memahami dasar, kamu bisa memperdalam struktur data, algoritma, OOP, testing, dan library.

Web development

Python dapat digunakan untuk pengembangan web melalui framework dan library yang sesuai.

Data analysis

Python banyak dipakai untuk mengolah, membersihkan, dan menganalisis data.

Automation

Script Python dapat membantu mengotomatisasi pekerjaan berulang yang aman dan sesuai kebutuhan.

AI dan machine learning

Python menjadi salah satu bahasa populer untuk eksperimen dan pengembangan aplikasi machine learning.

API

API memungkinkan program berkomunikasi dengan layanan atau aplikasi lain melalui antarmuka yang disediakan.

Database

Program dapat menyimpan data secara terstruktur menggunakan database dan query.

Portofolio

Proyek nyata membantu menunjukkan kemampuan. Mulai dari proyek kecil yang selesai dan bisa dijelaskan dengan baik.

Latihan Bab 18

Kerjakan setelah memahami seluruh materi bab ini. Cobalah menulis kode sendiri sebelum melihat referensi lain.

1. Pilih satu bidang Python yang ingin kamu pelajari dan jelaskan alasannya.
2. Buat daftar tiga proyek kecil untuk portofolio.
3. Tentukan konsep dasar yang masih perlu kamu kuasai.
4. Buat rencana belajar 7 hari yang realistis.
5. Tulis ide proyek akhir yang menggabungkan minimal tiga bab dalam ebook ini.

Ringkasan Bab 18

Gunakan halaman ini sebagai checkpoint. Jangan hanya menghafal sintaks; pastikan kamu bisa menjelaskan kapan sebuah konsep digunakan dan apa masalah yang diselesaikannya.

- Pahami konsep sebelum menghafal kode.
- Coba ubah contoh dengan nilai dan nama yang berbeda.
- Baca pesan error sebelum mencari solusi.
- Biasakan menulis kode sedikit demi sedikit lalu menjalankannya.
- Simpan proyek kecil agar perkembanganmu terlihat.

Catatan pribadi

Tulis konsep yang paling kamu pahami, konsep yang masih membingungkan, dan satu ide program yang ingin kamu coba.

Cheat Sheet Python Pemula

Sintaks yang sering dipakai

```
print("teks")
variabel = nilai
input("Pertanyaan: ")
if kondisi:
...
else:
...
for item in data:
...
while kondisi:
...
def nama_fungsi(parameter):
return hasil
import module
with open("file.txt", "r", encoding="utf-8") as file:
isi = file.read()
```

Checklist kemampuan

- Saya memahami apa itu program dan Python.
- Saya bisa membuat dan menjalankan file .py.
- Saya memahami variabel dan tipe data dasar.
- Saya bisa menggunakan operator dan input.
- Saya bisa membuat if/elif/else.
- Saya bisa membuat for dan while.
- Saya memahami list, tuple, set, dan dictionary.
- Saya bisa membuat fungsi.
- Saya bisa membaca/menulis file sederhana.
- Saya tahu cara membaca dan menangani error.
- Saya memahami dasar module, package, dan OOP.
- Saya mampu membuat proyek Python kecil.

Penutup

Selamat! Jika kamu sampai di halaman ini, kamu sudah memiliki peta dasar Python dari konsep pemrograman sampai proyek sederhana. Jangan merasa harus langsung menguasai semuanya. Pemrograman adalah keterampilan yang tumbuh melalui praktik.

Langkah terbaik berikutnya adalah memilih satu proyek kecil, menyelesaikannya, lalu memperbaikinya sedikit demi sedikit. Misalnya kalkulator, pengelola catatan, program kuis, konverter satuan, atau aplikasi sederhana berbasis data.

Ingat: programmer bukan orang yang selalu tahu jawabannya. Programmer adalah orang yang mau memecah masalah, mencoba, membaca error, mencari penyebab, dan memperbaiki program.

HANZZY DEV

Website: hanie-alfarisy-dev.vercel.app

Referensi utama: Dokumentasi resmi Python 3.14. [\[cite\]turn0view0](#)

Apendiks A — Tabel Tipe Data Python

Bagian ini menjadi referensi cepat setelah materi utama. Gunakan sebagai pengingat, bukan pengganti pemahaman konsep.

Istilah / Bagian	Penjelasan
int	Bilangan bulat, contoh 10 atau -3
float	Bilangan pecahan, contoh 3.14
str	Teks, contoh "Python"
bool	True atau False
list	Kumpulan berurutan yang dapat diubah
tuple	Kumpulan berurutan yang umumnya tidak diubah
set	Kumpulan nilai unik
dict	Pasangan key-value
None	Menandakan tidak ada nilai

Catatan: contoh di atas merangkum konsep dasar. Untuk perilaku detail dan fitur lengkap, gunakan dokumentasi resmi Python.

Apendiks B — Operator Penting

Bagian ini menjadi referensi cepat setelah materi utama. Gunakan sebagai pengingat, bukan pengganti pemahaman konsep.

Istilah / Bagian	Penjelasan
+ - * /	Penjumlahan, pengurangan, perkalian, pembagian
//	Pembagian bulat
%	Sisa pembagian
**	Pangkat
== != < > <= >=	Perbandingan
and / or / not	Logika
= += -= *= /=	Assignment

Catatan: contoh di atas merangkum konsep dasar. Untuk perilaku detail dan fitur lengkap, gunakan dokumentasi resmi Python.

Apendiks C — Method String

Bagian ini menjadi referensi cepat setelah materi utama. Gunakan sebagai pengingat, bukan pengganti pemahaman konsep.

Istilah / Bagian	Penjelasan
<code>lower()</code>	Mengubah teks menjadi huruf kecil
<code>upper()</code>	Mengubah teks menjadi huruf besar
<code>strip()</code>	Menghapus whitespace di ujung
<code>replace(a,b)</code>	Mengganti bagian teks
<code>split()</code>	Memecah string
<code>join()</code>	Menggabungkan string
<code>startswith()</code>	Mengecek awalan
<code>endswith()</code>	Mengecek akhiran

Catatan: contoh di atas merangkum konsep dasar. Untuk perilaku detail dan fitur lengkap, gunakan dokumentasi resmi Python.

Apendiks D — Method List

Bagian ini menjadi referensi cepat setelah materi utama. Gunakan sebagai pengingat, bukan pengganti pemahaman konsep.

Istilah / Bagian	Penjelasan
<code>append(x)</code>	Menambah satu item
<code>extend(data)</code>	Menambah banyak item
<code>insert(i,x)</code>	Menyisipkan item
<code>remove(x)</code>	Menghapus berdasarkan nilai
<code>pop()</code>	Menghapus dan mengambil item
<code>sort()</code>	Mengurutkan list
<code>reverse()</code>	Membalik urutan
<code>len(data)</code>	Menghitung jumlah item

Catatan: contoh di atas merangkum konsep dasar. Untuk perilaku detail dan fitur lengkap, gunakan dokumentasi resmi Python.

Apendiks E — Pola Percabangan

Bagian ini menjadi referensi cepat setelah materi utama. Gunakan sebagai pengingat, bukan pengganti pemahaman konsep.

Istilah / Bagian	Penjelasan
if	Jalankan jika kondisi benar
if/else	Dua jalur
if/elif/else	Beberapa pilihan
nested if	Kondisi di dalam kondisi
and	Semua kondisi harus benar
or	Salah satu kondisi cukup benar
not	Membalik nilai boolean

Catatan: contoh di atas merangkum konsep dasar. Untuk perilaku detail dan fitur lengkap, gunakan dokumentasi resmi Python.

Apendiks F — Pola Perulangan

Bagian ini menjadi referensi cepat setelah materi utama. Gunakan sebagai pengingat, bukan pengganti pemahaman konsep.

Istilah / Bagian	Penjelasan
<code>for item in data</code>	Mengunjungi item satu per satu
<code>for i in range(n)</code>	Mengulang berdasarkan rentang
<code>while kondisi</code>	Mengulang selama kondisi benar
<code>break</code>	Keluar dari loop
<code>continue</code>	Lanjut ke iterasi berikutnya

Catatan: contoh di atas merangkum konsep dasar. Untuk perilaku detail dan fitur lengkap, gunakan dokumentasi resmi Python.

Apendiks G — Fungsi

Bagian ini menjadi referensi cepat setelah materi utama. Gunakan sebagai pengingat, bukan pengganti pemahaman konsep.

Istilah / Bagian	Penjelasan
<code>def nama():</code>	Mendefinisikan fungsi
parameter	Data yang diterima fungsi
<code>return</code>	Mengembalikan hasil
default argument	Nilai bawaan parameter
scope	Wilayah akses nama

Catatan: contoh di atas merangkum konsep dasar. Untuk perilaku detail dan fitur lengkap, gunakan dokumentasi resmi Python.

Apendiks H — File

Bagian ini menjadi referensi cepat setelah materi utama. Gunakan sebagai pengingat, bukan pengganti pemahaman konsep.

Istilah / Bagian	Penjelasan
<code>open(path,'r')</code>	Membaca
<code>open(path,'w')</code>	Menulis ulang
<code>open(path,'a')</code>	Menambahkan
<code>with open(...)</code>	Mengelola file dengan aman
<code>read()</code>	Membaca seluruh isi
<code>write()</code>	Menulis teks

Catatan: contoh di atas merangkum konsep dasar. Untuk perilaku detail dan fitur lengkap, gunakan dokumentasi resmi Python.

Apendiks I — Exception Umum

Bagian ini menjadi referensi cepat setelah materi utama. Gunakan sebagai pengingat, bukan pengganti pemahaman konsep.

Istilah / Bagian	Penjelasan
SyntaxError	Penulisan sintaks tidak valid
NameError	Nama tidak ditemukan
TypeError	Operasi tidak cocok dengan tipe
ValueError	Nilai tidak sesuai
IndexError	Indeks di luar batas
KeyError	Key dictionary tidak ditemukan
ZeroDivisionError	Membagi dengan nol

Catatan: contoh di atas merangkum konsep dasar. Untuk perilaku detail dan fitur lengkap, gunakan dokumentasi resmi Python.

Apendiks J — Modul Standard Library

Bagian ini menjadi referensi cepat setelah materi utama. Gunakan sebagai pengingat, bukan pengganti pemahaman konsep.

Istilah / Bagian	Penjelasan
math	Matematika
random	Bilangan/pilihan acak
datetime	Tanggal dan waktu
json	Membaca/menulis JSON
pathlib	Mengelola path file
os	Fungsi terkait sistem operasi
statistics	Statistik dasar

Catatan: contoh di atas merangkum konsep dasar. Untuk perilaku detail dan fitur lengkap, gunakan dokumentasi resmi Python.

Apendiks K — Contoh Mini 1: Konverter Suhu

Bagian ini menjadi referensi cepat setelah materi utama. Gunakan sebagai pengingat, bukan pengganti pemahaman konsep.

Istilah / Bagian	Penjelasan
Input	Suhu Celsius dari pengguna
Proses	Gunakan rumus Celsius ke Fahrenheit
Output	Tampilkan hasil
Konsep	input, float, operasi, print

Catatan: contoh di atas merangkum konsep dasar. Untuk perilaku detail dan fitur lengkap, gunakan dokumentasi resmi Python.

Apendiks L — Contoh Mini 2: Cek Bilangan

Bagian ini menjadi referensi cepat setelah materi utama. Gunakan sebagai pengingat, bukan pengganti pemahaman konsep.

Istilah / Bagian	Penjelasan
Input	Satu angka
Proses	Gunakan % 2
Output	Genap atau ganjil
Konsep	input, int, if, operator

Catatan: contoh di atas merangkum konsep dasar. Untuk perilaku detail dan fitur lengkap, gunakan dokumentasi resmi Python.

Apendiks M — Contoh Mini 3: Daftar Belanja

Bagian ini menjadi referensi cepat setelah materi utama. Gunakan sebagai pengingat, bukan pengganti pemahaman konsep.

Istilah / Bagian	Penjelasan
Input	Nama barang
Proses	Simpan ke list
Output	Tampilkan seluruh item
Konsep	list, append, loop

Catatan: contoh di atas merangkum konsep dasar. Untuk perilaku detail dan fitur lengkap, gunakan dokumentasi resmi Python.

Apendiks N — Contoh Mini 4: Nilai Siswa

Bagian ini menjadi referensi cepat setelah materi utama. Gunakan sebagai pengingat, bukan pengganti pemahaman konsep.

Istilah / Bagian	Penjelasan
Input	Beberapa nilai
Proses	Hitung rata-rata dan kategori
Output	Tampilkan hasil
Konsep	list, sum, len, if

Catatan: contoh di atas merangkum konsep dasar. Untuk perilaku detail dan fitur lengkap, gunakan dokumentasi resmi Python.

Apendiks O — Contoh Mini 5: Menu Program

Bagian ini menjadi referensi cepat setelah materi utama. Gunakan sebagai pengingat, bukan pengganti pemahaman konsep.

Istilah / Bagian	Penjelasan
Input	Pilihan menu
Proses	Percabangan + fungsi
Output	Hasil sesuai pilihan
Konsep	while, if, function

Catatan: contoh di atas merangkum konsep dasar. Untuk perilaku detail dan fitur lengkap, gunakan dokumentasi resmi Python.

Apendiks P — Checklist Debugging

Bagian ini menjadi referensi cepat setelah materi utama. Gunakan sebagai pengingat, bukan pengganti pemahaman konsep.

Istilah / Bagian	Penjelasan
1	Baca error/traceback
2	Cari baris yang disebut
3	Periksa nilai variabel
4	Uji bagian kecil
5	Perbaiki satu masalah setiap kali
6	Jalankan kembali
7	Tambahkan test case

Catatan: contoh di atas merangkum konsep dasar. Untuk perilaku detail dan fitur lengkap, gunakan dokumentasi resmi Python.

Apendiks Q — Ide Proyek Portofolio

Bagian ini menjadi referensi cepat setelah materi utama. Gunakan sebagai pengingat, bukan pengganti pemahaman konsep.

Istilah / Bagian	Penjelasan
Pemula	Kalkulator, kuis, konverter, catatan
Menengah	To-do list, pengelola nilai, pencatat keuangan sederhana
Lanjutan	Aplikasi web, API client, analisis data, otomasi
Tips	Pilih proyek yang selesai dan bisa kamu jelaskan

Catatan: contoh di atas merangkum konsep dasar. Untuk perilaku detail dan fitur lengkap, gunakan dokumentasi resmi Python.

Apendiks R — Kebiasaan Programmer

Bagian ini menjadi referensi cepat setelah materi utama. Gunakan sebagai pengingat, bukan pengganti pemahaman konsep.

Istilah / Bagian	Penjelasan
Baca dokumentasi	Cari informasi dari sumber resmi
Eksperimen	Ubah contoh dan lihat hasil
Catat error	Simpan solusi yang sudah ditemukan
Gunakan Git	Pelajari version control saat proyek mulai serius
Refactor	Rapikan kode setelah program bekerja

Catatan: contoh di atas merangkum konsep dasar. Untuk perilaku detail dan fitur lengkap, gunakan dokumentasi resmi Python.

Apendiks S — Glosarium Python

Bagian ini menjadi referensi cepat setelah materi utama. Gunakan sebagai pengingat, bukan pengganti pemahaman konsep.

Istilah / Bagian	Penjelasan
Algorithm	Langkah sistematis menyelesaikan masalah
Argument	Nilai yang diberikan saat fungsi dipanggil
Bug	Kesalahan dalam program
Exception	Kejadian error saat program berjalan
Function	Blok kode yang dapat dipanggil
Iteration	Satu putaran loop
Module	File Python yang dapat diimpor
Object	Instance dari class
Package	Kumpulan module
Variable	Nama yang merujuk ke nilai

Catatan: contoh di atas merangkum konsep dasar. Untuk perilaku detail dan fitur lengkap, gunakan dokumentasi resmi Python.

Apendiks T — Rencana Praktik 14 Hari

Bagian ini menjadi referensi cepat setelah materi utama. Gunakan sebagai pengingat, bukan pengganti pemahaman konsep.

Istilah / Bagian	Penjelasan
Hari 1-2	print, variabel, tipe data
Hari 3	operator dan input
Hari 4	if/elif/else
Hari 5	for dan range
Hari 6	while
Hari 7	string
Hari 8	list dan tuple
Hari 9	set dan dictionary
Hari 10	fungsi
Hari 11	file/JSON
Hari 12	exception
Hari 13	modul dan OOP
Hari 14	buat proyek kecil

Catatan: contoh di atas merangkum konsep dasar. Untuk perilaku detail dan fitur lengkap, gunakan dokumentasi resmi Python.

Proyek Pendalaman 1 — Kalkulator Mini

Tujuan proyek ini adalah menggabungkan input, konversi tipe, fungsi, percabangan, loop, dan exception handling. Pecah program menjadi fungsi seperti tambah(), kurang(), kali(), dan bagi(). Menu utama bertugas meminta pilihan dan memanggil fungsi yang sesuai.

Contoh implementasi

```
def tambah(a, b):  
    return a + b  
  
def bagi(a, b):  
    if b == 0:  
        return None  
    return a / b  
  
print("Kalkulator Mini")  
a = float(input("Angka pertama: "))  
b = float(input("Angka kedua: "))  
print("Hasil tambah:", tambah(a, b))  
print("Hasil bagi:", bagi(a, b))
```

Cara mengembangkan proyek

Mulai dari versi paling kecil yang bisa berjalan. Setelah itu tambahkan satu fitur setiap kali, misalnya validasi input, menu, penyimpanan data, atau pemisahan fungsi. Dengan pendekatan ini kamu belajar merancang program, bukan hanya menyalin kode.

Proyek Pendalaman 2 — Kuis Sederhana

Kuis mengajarkan bagaimana data pertanyaan dan jawaban dapat disimpan lalu diproses dengan loop. Untuk versi lebih rapi, pertanyaan bisa disimpan sebagai dictionary atau list of dictionaries. Skor dapat dinaikkan ketika jawaban pengguna cocok dengan jawaban yang benar.

Contoh implementasi

```
soal = [
    {"tanya": "Bahasa ini bernama apa?", "jawab": "python"},
    {"tanya": "Berapa 2 + 3?", "jawab": "5"}
]

skor = 0
for item in soal:
    jawaban = input(item["tanya"] + " ").strip().lower()
    if jawaban == item["jawab"]:
        skor += 1

print("Skor:", skor, "/", len(soal))
```

Cara mengembangkan proyek

Mulai dari versi paling kecil yang bisa berjalan. Setelah itu tambahkan satu fitur setiap kali, misalnya validasi input, menu, penyimpanan data, atau pemisahan fungsi. Dengan pendekatan ini kamu belajar merancang program, bukan hanya menyalin kode.

Proyek Pendalaman 3 — Catatan Teks

Program catatan sederhana dapat menyimpan input pengguna ke file. Konsep utamanya adalah input, mode append, with open, dan pembacaan file. Setelah versi dasar selesai, kamu dapat menambahkan menu untuk melihat, menambah, dan menghapus catatan.

Contoh implementasi

```
catatan = input("Tulis catatan: ")

with open("catatan.txt", "a", encoding="utf-8") as file:
    file.write(catatan + "\n")

with open("catatan.txt", "r", encoding="utf-8") as file:
    isi = file.read()

print("\nSemua catatan:")
print(isi)
```

Cara mengembangkan proyek

Mulai dari versi paling kecil yang bisa berjalan. Setelah itu tambahkan satu fitur setiap kali, misalnya validasi input, menu, penyimpanan data, atau pemisahan fungsi. Dengan pendekatan ini kamu belajar merancang program, bukan hanya menyalin kode.

Proyek Pendalaman 4 — Penyimpanan JSON

JSON cocok untuk menyimpan data terstruktur sederhana. Python memiliki modul json. Pola umum yang perlu dipahami adalah dumps/dump untuk serialisasi dan loads/load untuk membaca kembali JSON.

Contoh implementasi

```
import json

data = {
    "nama": "Budi",
    "kelas": "X",
    "hobi": ["coding", "membaca"]
}

with open("data.json", "w", encoding="utf-8") as file:
    json.dump(data, file, indent=2)

with open("data.json", "r", encoding="utf-8") as file:
    data_baru = json.load(file)

print(data_baru)
```

Cara mengembangkan proyek

Mulai dari versi paling kecil yang bisa berjalan. Setelah itu tambahkan satu fitur setiap kali, misalnya validasi input, menu, penyimpanan data, atau pemisahan fungsi. Dengan pendekatan ini kamu belajar merancang program, bukan hanya menyalin kode.